

CNPJ Data Publisher

Documento de implementação E2E

1. Objetivo

Implementar serviço independente chamado:

```
cnpj-data-publisher
```

Responsabilidades:

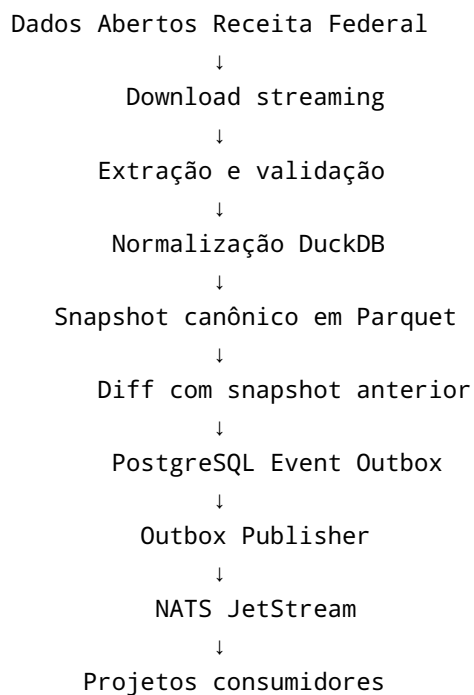
1. Descobrir snapshot mensal mais recente dos Dados Abertos do CNPJ.
2. Baixar arquivos necessários.
3. Validar integridade dos downloads.
4. Extrair e tratar dados.
5. Normalizar empresas e estabelecimentos.
6. Gerar snapshot canônico em Parquet.
7. Identificar empresas ativas.
8. Comparar snapshot atual com anterior.
9. Detectar empresas novas, alteradas, reativadas e inativadas.
10. Publicar eventos no NATS JetStream.
11. Disponibilizar backfill controlado de empresas ativas.
12. Rodar via Docker.
13. Possuir Helm Chart pronto para K3s.

Projeto não deve:

```
descobrir domínios  
executar BBot  
buscar redes sociais  
validar e-mails  
calcular lead score  
conhecer ShadowTrace  
executar enriquecimento
```

Integração com consumidores acontece somente por eventos NATS JetStream.

2. Arquitetura



Componentes executáveis:

```
ingestor  
outbox-publisher  
backfill-publisher  
migrations
```

Usar mesma imagem Docker com comandos diferentes.

Exemplo:

```
cnpj-data-publisher ingest --snapshot latest  
cnpj-data-publisher publish-outbox  
cnpj-data-publisher backfill --snapshot latest  
cnpj-data-publisher migrate
```

3. Stack

```
Python 3.12
DuckDB
PyArrow
Parquet
PostgreSQL 16
SQLAlchemy 2
Alembic
Pydantic v2
Typer
httpx
nats-py
structlog
Prometheus Client
OpenTelemetry opcional
pytest
testcontainers
Docker
Helm
K3s
```

DuckDB processa arquivos grandes.

Parquet armazena snapshots canônicos.

PostgreSQL armazena:

```
execuções
manifestos
estado do pipeline
outbox
checkpoints
falhas
backfills
```

Não armazenar obrigatoriamente 25 milhões de empresas no PostgreSQL.

4. Estrutura do repositório

```
cnpj-data-publisher/  
├── src/  
│   ├── cnpj_data_publisher/  
│   │   ├── cli.py  
│   │   ├── config.py  
│   │   ├── logging.py  
│   │   ├── metrics.py  
│   │   ├── database/  
│   │   │   ├── models.py  
│   │   │   ├── repositories.py  
│   │   │   └── session.py  
│   │   ├── receita/  
│   │   │   ├── discovery.py  
│   │   │   ├── downloader.py  
│   │   │   ├── extractor.py  
│   │   │   ├── manifest.py  
│   │   │   └── layouts/  
│   │   │       ├── empresas.py  
│   │   │       ├── estabelecimentos.py  
│   │   │       ├── simples.py  
│   │   │       ├── socios.py  
│   │   │       └── lookup_tables.py  
│   │   ├── processing/  
│   │   │   ├── normalizer.py  
│   │   │   ├── canonical_snapshot.py  
│   │   │   ├── fingerprints.py  
│   │   │   ├── diff.py  
│   │   │   └── validation.py  
│   │   ├── events/  
│   │   │   ├── envelope.py  
│   │   │   ├── company_events.py  
│   │   │   ├── schemas/  
│   │   │   └── subjects.py  
│   │   ├── outbox/  
│   │   │   ├── service.py  
│   │   │   └── publisher.py  
│   │   ├── backfill/  
│   │   │   ├── service.py  
│   │   │   └── checkpoints.py  
│   │   └── storage/  
│   │       ├── local.py  
│   │       └── s3.py  
└── migrations/
```

```
├─ helm/
│   └─ cnpj-data-publisher/
│       ├── Chart.yaml
│       ├── Chart.lock
│       ├── values.yaml
│       ├── values-k3s.yaml
│       ├── values-production.yaml.example
│       ├── charts/
│       └─ templates/
├─ docker/
├─ scripts/
├─ tests/
│   ├── fixtures/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └─ performance/
├─ contracts/
│   ├── company-discovered-v1.schema.json
│   ├── company-updated-v1.schema.json
│   ├── company-reactivated-v1.schema.json
│   ├── company-inactivated-v1.schema.json
│   ├── snapshot-ready-v1.schema.json
│   └─ examples/
├─ Dockerfile
├─ docker-compose.yml
├─ Makefile
├─ pyproject.toml
├─ .env.example
└─ README.md
```

5. Modos de execução

5.1 Ingestão mensal

```
cnpj-data-publisher ingest --snapshot latest
```

Etapas:

```
DISCOVERING
DOWNLOADING
```

```
VALIDATING_DOWNLOADS  
EXTRACTING  
NORMALIZING  
BUILDING_SNAPSHOT  
VALIDATING_SNAPSHOT  
CALCULATING_DIFF  
CREATING_EVENTS  
COMPLETED
```

Permitir snapshot específico:

```
cnpj-data-publisher ingest --snapshot 2026-07
```

5.2 Outbox publisher

```
cnpj-data-publisher publish-outbox
```

Processo contínuo.

Responsável somente por publicar eventos pendentes no NATS JetStream.

5.3 Backfill

```
cnpj-data-publisher backfill --snapshot latest
```

Publica empresas ativas do snapshot selecionado.

Deve permitir:

```
cnpj-data-publisher backfill  
  --snapshot 2026-07  
  --state SP  
  --main-cnae 6201501  
  --rate 100
```

Filtros permitidos:

```
UF  
CNAE principal  
data de abertura inicial
```

```
data de abertura final  
matriz ou filial  
porte  
opção MEI  
opção Simples
```

Backfill deve ser resumível.

5.4 Migrations

```
cnpj-data-publisher migrate
```

6. Descoberta dos snapshots

Implementar cliente que:

1. Consulta diretório público dos snapshots.
2. Descobre meses disponíveis.
3. Ordena versões.
4. Identifica snapshot mais recente completo.
5. Ignora snapshot já processado com sucesso.
6. Permite reproprocessamento forçado.

Configuração:

```
RECEITA_BASE_URL=  
RECEITA_REQUEST_TIMEOUT_SECONDS=60  
RECEITA_MAX_DOWNLOAD_CONCURRENCY=4  
RECEITA_DOWNLOAD_RETRIES=5  
RECEITA_DOWNLOAD_CHUNK_SIZE_BYTES=8388608
```

Comando forçado:

```
cnpj-data-publisher ingest --snapshot 2026-07 --force
```

Não considerar snapshot válido antes de arquivos mínimos estarem disponíveis.

Arquivos mínimos:

```
Empresas*.zip
Estabelecimentos*.zip
Cnaes.zip
Municipios.zip
Naturezas*.zip
Países*.zip
Qualificacoes*.zip
Simples.zip
```

Sócios opcionais:

```
RECEITA_INCLUDE_PARTNERS=false
```

Quando ativo:

```
Socios*.zip
```

7. Download

Requisitos:

```
streaming
retry exponencial
timeout
limite de concorrência
download parcial
SHA-256
ETag
Last-Modified
tamanho esperado
tamanho baixado
manifesto persistido
```

Não carregar ZIP inteiro em memória.

Estrutura:

```
/data/
├─ downloads/
```

```
|   └─ 2026-07/
|─ extracted/
|   └─ 2026-07/
|─ snapshots/
|   └─ 2026-07/
|─ diffs/
|   └─ 2026-07/
|─ rejected/
|   └─ 2026-07/
└─ temporary/
```

Arquivo baixado deve usar escrita atômica:

```
arquivo.zip.partial
arquivo.zip
```

Renomear somente depois de download e validação.

Manifesto:

```
{
  "snapshot_version": "2026-07",
  "files": [
    {
      "name": "Empresas0.zip",
      "url": "...",
      "size": 123456789,
      "etag": "...",
      "last_modified": "...",
      "sha256": "...",
      "downloaded_at": "..."
    }
  ]
}
```

8. Extração

Requisitos:

```
extração streaming quando possível
proteção contra Zip Slip
```

```
limite de tamanho extraído  
detecção de arquivo corrompido  
registro de arquivos extraídos  
limpeza após falha
```

Não confiar em caminhos internos do ZIP.

Bloquear:

```
../  
caminho absoluto  
symlink inesperado
```

Configuração:

```
MAX_EXTRACTED_FILE_SIZE_BYTES=50000000000  
MAX_TOTAL_EXTRACTED_SIZE_BYTES=500000000000
```

9. Parsing dos arquivos

Layout centralizado.

Não usar índices mágicos espalhados.

Exemplo:

```
ESTABELECIMENTOS_COLUMNS = [  
    "cnpj_basic",  
    "cnpj_order",  
    "cnpj_check_digits",  
    "headquarters_branch",  
    "trade_name",  
    "registration_status_code",  
    "registration_status_date",  
    "registration_status_reason",  
    "foreign_city",  
    "country_code",  
    "opening_date",  
    "main_cnae",  
    "secondary_cnaes",  
    "street_type",
```

```
"street",  
"number",  
"complement",  
"district",  
"postal_code",  
"state",  
"city_code",  
"area_code_1",  
"phone_1",  
"area_code_2",  
"phone_2",  
"fax_area_code",  
"fax",  
"email",  
"special_status",  
"special_status_date"  
]
```

Parâmetros:

```
delimiter=";"  
encoding="latin-1"  
quotechar="'"
```

Tratar:

```
string vazia  
datas inválidas  
capital social com vírgula  
linhas incompletas  
linhas extras  
caracteres inválidos  
e-mails malformados  
telefones incompletos  
CNPJ numérico ou alfanumérico
```

CNPJ sempre como texto.

Nunca converter para inteiro.

10. Modelo canônico

Cada estabelecimento deve gerar registro canônico.

```
{
  "cnpj": "12345678000190",
  "cnpj_basic": "12345678",
  "cnpj_order": "0001",
  "cnpj_check_digits": "90",
  "headquarters_branch": "HEADQUARTERS",
  "legal_name": "EMPRESA EXEMPLO LTDA",
  "trade_name": "EMPRESA EXEMPLO",
  "registration_status": "ACTIVE",
  "registration_status_code": "02",
  "registration_status_date": "2026-07-01",
  "registration_status_reason": null,
  "opening_date": "2026-06-10",
  "legal_nature_code": "2062",
  "legal_nature_description": "SOCIEDADE EMPRESARIA LIMITADA",
  "main_cnae": "6201501",
  "main_cnae_description": "...",
  "secondary_cnaes": [
    "6202300"
  ],
  "company_size_code": "03",
  "share_capital": "100000.00",
  "simple_tax_option": true,
  "mei_option": false,
  "address": {
    "street_type": "RUA",
    "street": "EXEMPLO",
    "number": "100",
    "complement": null,
    "district": "CENTRO",
    "postal_code": "00000000",
    "city_code": "7107",
    "city_name": "SAO PAULO",
    "state": "SP",
    "country_code": "105"
  },
  "contacts": {
    "phones": [
      {
        "area_code": "11",
        "number": "999999999"
      }
    ]
  }
}
```

```

    ],
    "fax": null,
    "email": "CONTATO@EMPRESA.COM.BR"
  },
  "source": {
    "dataset": "RECEITA_FEDERAL_CNPJ",
    "snapshot_version": "2026-07"
  }
}

```

Normalizações:

```

trim
remoção de caracteres de controle
uppercase em campos cadastrais
lowercase em e-mail
ordenação de CNAEs secundários
padronização de null
padronização de datas ISO-8601
capital social Decimal
telefone sem pontuação
CEP sem pontuação

```

Não remover acentos de campos exibidos.

Criar campos auxiliares normalizados somente quando necessário.

11. Situação cadastral

Mapear códigos para nomes.

```

01 = NULL
02 = ACTIVE
03 = SUSPENDED
04 = UNFIT
08 = CLOSED
UNKNOWN = UNKNOWN

```

Empresa ativa:

```
registration_status_code = "02"
```

Matriz:

```
headquarters_branch = "1"
```

Configuração:

```
INCLUDE_BRANCHES=false
```

Quando falso, snapshot de publicação contém somente matrizes.

Snapshot bruto canônico pode manter matrizes e filiais.

12. Snapshot Parquet

Gerar snapshot canônico particionado.

Estrutura:

```
/data/snapshots/2026-07/  
├─ manifest.json  
├─ all/  
│   ├── state=SP/  
│   ├── state=RJ/  
│   └─ state=MG/  
└─ active/  
    ├── state=SP/  
    ├── state=RJ/  
    └─ state=MG/
```

Formato:

```
Parquet  
compressão ZSTD  
row groups configuráveis  
schema fixo  
partição por UF
```

Configuração:

```
PARQUET_COMPRESSION=zstd
PARQUET_ROW_GROUP_SIZE=250000
```

Manifesto:

```
{
  "snapshot_version": "2026-07",
  "schema_version": 1,
  "created_at": "...",
  "total_rows": 0,
  "active_rows": 0,
  "headquarters_rows": 0,
  "branches_rows": 0,
  "rejected_rows": 0,
  "partitions": [],
  "fingerprint": "sha256"
}
```

Publicar snapshot somente após validação completa.

Usar diretório temporário:

```
/data/snapshots/2026-07-building/
```

Depois:

```
rename atômico para /data/snapshots/2026-07/
```

13. Validação do snapshot

Antes do diff:

```
arquivos mínimos encontrados
nenhuma partição vazia inesperada
CNPJ não vazio
CNPJ único por estabelecimento
situação cadastral válida
```

```
datas parseadas
contagem mínima plausível
percentual de rejeição abaixo do limite
Parquet legível
manifesto completo
```

Configuração:

```
MAX_REJECTED_ROW_PERCENTAGE=0.01
MIN_EXPECTED_TOTAL_ROWS=1000000
```

Quando validação falhar:

```
não substituir snapshot anterior
não gerar diff
não criar eventos
marcar execução FAILED
preservar arquivos para diagnóstico
```

14. Fingerprint

Criar fingerprint determinístico por empresa.

Campos:

```
cnpj
headquarters_branch
legal_name
trade_name
registration_status_code
registration_status_date
opening_date
legal_nature_code
main_cnae
secondary_cnaes
company_size_code
share_capital
simple_tax_option
mei_option
```

```
address
contacts
```

Processo:

```
normalizar
serializar JSON canônico
ordenar chaves
ordenar listas
SHA-256
```

Exemplo:

```
payload = json.dumps(
    fingerprint_data,
    ensure_ascii=False,
    sort_keys=True,
    separators=(",", ":"),
)

fingerprint = hashlib.sha256(payload.encode("utf-8")).hexdigest()
```

Adicionar fingerprint ao Parquet.

15. Diff entre snapshots

Comparar:

```
snapshot atual
snapshot anterior concluído
```

Usar DuckDB.

Eventos:

```
COMPANY_DISCOVERED
COMPANY_UPDATED
COMPANY_REACTIVATED
COMPANY_INACTIVATED
```

Regras:

```
não existia antes e agora está ativa
→ COMPANY_DISCOVERED

existia ativa, continua ativa e fingerprint mudou
→ COMPANY_UPDATED

existia não ativa e agora está ativa
→ COMPANY_REACTIVATED

existia ativa e agora não está ativa
→ COMPANY_INACTIVATED

ativa e fingerprint igual
→ nenhum evento

não ativa antes e continua não ativa
→ nenhum evento
```

Não tratar ausência isolada como inativação sem registro atual correspondente.

Inativação deve depender da situação cadastral atual diferente de 02.

Gerar arquivos de diff:

```
/data/diffs/2026-07/
├─ discovered.parquet
├─ updated.parquet
├─ reactivated.parquet
├─ inactivated.parquet
└─ manifest.json
```

Manifesto:

```
{
  "snapshot_version": "2026-07",
  "previous_snapshot_version": "2026-06",
  "discovered": 0,
  "updated": 0,
  "reactivated": 0,
  "inactivated": 0,
```

```
"unchanged": 0  
}
```

16. Primeiro snapshot

Primeira execução não deve publicar automaticamente milhões de eventos.

Configuração:

```
INITIAL_SNAPSHOT_MODE=STORE_ONLY
```

Valores:

```
STORE_ONLY  
PUBLISH_ALL
```

Padrão:

```
STORE_ONLY
```

STORE_ONLY :

```
baixa  
normaliza  
gera snapshot  
registra execução  
publica somente SNAPSHOT_READY
```

PUBLISH_ALL :

```
cria backfill automático de todas empresas ativas  
usa rate limit  
usa checkpoint  
pode demorar vários dias
```

Recomendação:

```
primeiro snapshot em STORE_ONLY  
backfill iniciado manualmente  
deltas mensais publicados automaticamente
```

17. Bulk bootstrap

Para consumidores que precisam de todas empresas ativas, oferecer duas opções.

Opção A — Snapshot Parquet

Publicar evento:

```
company.br.cnpj.snapshot.ready.v1
```

Evento contém:

```
versão  
manifesto  
localização do snapshot  
checksums  
contagem de registros  
schema version
```

Consumidor baixa e processa Parquet.

Melhor opção para milhões de registros.

Opção B — Backfill JetStream

Publicar eventos individuais com limite de taxa.

Comando:

```
cnpj-data-publisher backfill  
  --snapshot 2026-07  
  --rate 200  
  --batch-size 1000
```

Backfill deve:

```
criar execução
registrar filtros
ler Parquet em ordem determinística
publicar por lotes
salvar cursor
retomar após restart
impedir duplicação
permitir pause
permitir cancelamento
```

18. PostgreSQL

Criar migrations Alembic.

18.1 source_snapshots

```
id UUID PK
snapshot_version VARCHAR UNIQUE NOT NULL
status VARCHAR NOT NULL
source_url TEXT
schema_version INTEGER NOT NULL
manifest JSONB
download_started_at TIMESTAMPTZ
download_completed_at TIMESTAMPTZ
processing_started_at TIMESTAMPTZ
processing_completed_at TIMESTAMPTZ
previous_snapshot_version VARCHAR NULL
error_code VARCHAR NULL
error_message TEXT NULL
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

Status:

```
DISCOVERED
DOWNLOADING
DOWNLOADED
EXTRACTING
NORMALIZING
VALIDATING
DIFFING
```

```
CREATING_EVENTS  
COMPLETED  
FAILED
```

18.2 ingest_runs

```
id UUID PK  
snapshot_id UUID FK  
status VARCHAR  
started_at TIMESTAMPTZ  
completed_at TIMESTAMPTZ NULL  
statistics JSONB  
configuration JSONB  
error_code VARCHAR NULL  
error_message TEXT NULL  
created_at TIMESTAMPTZ
```

18.3 event_outbox

```
id UUID PK  
event_id UUID UNIQUE NOT NULL  
subject VARCHAR NOT NULL  
event_type VARCHAR NOT NULL  
event_version INTEGER NOT NULL  
aggregate_id VARCHAR NOT NULL  
payload JSONB NOT NULL  
headers JSONB NOT NULL  
status VARCHAR NOT NULL  
attempts INTEGER NOT NULL DEFAULT 0  
available_at TIMESTAMPTZ NOT NULL  
published_at TIMESTAMPTZ NULL  
last_error TEXT NULL  
created_at TIMESTAMPTZ NOT NULL  
updated_at TIMESTAMPTZ NOT NULL
```

Status:

```
PENDING  
PUBLISHING  
PUBLISHED  
FAILED
```

Índices:

```
(status, available_at)
aggregate_id
event_type
created_at
```

18.4 backfill_runs

```
id UUID PK
snapshot_version VARCHAR NOT NULL
status VARCHAR NOT NULL
filters JSONB
rate_limit INTEGER
batch_size INTEGER
cursor JSONB
total_rows BIGINT
processed_rows BIGINT
published_rows BIGINT
failed_rows BIGINT
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ NULL
error_message TEXT NULL
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

Status:

```
PENDING
RUNNING
PAUSED
COMPLETED
FAILED
CANCELLED
```

18.5 rejected_rows

```
id UUID PK
snapshot_version VARCHAR NOT NULL
source_file TEXT NOT NULL
line_number BIGINT
raw_data TEXT
error_code VARCHAR
```

```
error_message TEXT
created_at TIMESTAMPTZ
```

18.6 system_locks

```
lock_name VARCHAR PK
owner_id VARCHAR
acquired_at TIMESTAMPTZ
expires_at TIMESTAMPTZ
```

Usar PostgreSQL advisory lock quando possível.

Impedir duas ingestões simultâneas.

19. Outbox Pattern

Eventos devem ser persistidos antes de publicação.

Fluxo:

```
diff
↓
criação de event_outbox
↓
commit PostgreSQL
↓
outbox publisher
↓
JetStream ACK
↓
status PUBLISHED
```

Nunca publicar diretamente no NATS durante cálculo do diff.

Outbox publisher:

```
SELECT *
FROM event_outbox
WHERE status = 'PENDING'
AND available_at <= NOW()
ORDER BY created_at
```

```
FOR UPDATE SKIP LOCKED
LIMIT :batch_size;
```

Configuração:

```
OUTBOX_BATCH_SIZE=500
OUTBOX_CONCURRENCY=20
OUTBOX_POLL_INTERVAL_MS=1000
OUTBOX_MAX_ATTEMPTS=20
OUTBOX_RETRY_BASE_SECONDS=5
OUTBOX_RETRY_MAX_SECONDS=3600
```

Ao publicar:

```
aguardar PubAck
marcar PUBLISHED
registrar stream
registrar sequence
```

Usar:

```
Nats-Msg-Id = event_id
```

20. NATS JetStream

20.1 Stream

Nome:

```
BRAZIL_COMPANY_EVENTS
```

Subjects:

```
company.br.cnpj.>
```

Configuração:

```
storage: file
retention: limits
discard: old
duplicate_window: 24h
max_age: 30d
replicas: configurável
```

K3s single-node:

```
replicas: 1
```

K3s multi-node:

```
replicas: 3
```

20.2 Subjects

```
company.br.cnpj.discovered.v1
company.br.cnpj.updated.v1
company.br.cnpj.reactivated.v1
company.br.cnpj.inactivated.v1

company.br.cnpj.snapshot.ready.v1
company.br.cnpj.ingest.completed.v1
company.br.cnpj.ingest.failed.v1
```

Projeto possui permissão somente para publicar nesses subjects.

Não deve consumir eventos de enriquecimento.

21. Contrato de evento

Envelope padrão:

```
{
  "event_id": "uuid",
  "event_type": "COMPANY_DISCOVERED",
  "event_version": 1,
  "subject": "company.br.cnpj.discovered.v1",
```

```
{
  "source": "cnpj-data-publisher",
  "occurred_at": "2026-08-05T20:00:00Z",
  "correlation_id": "uuid",
  "causation_id": null,
  "trace_id": null,
  "data": {},
  "metadata": {}
}
```

Campos obrigatórios:

```
event_id
event_type
event_version
subject
source
occurred_at
correlation_id
data
metadata
```

JSON Schema deve ser fonte de verdade.

Nunca alterar `v1` de forma incompatível.

Criar `v2` quando contrato mudar.

22. Evento COMPANY_DISCOVERED

```
{
  "event_id": "018f0000-0000-7000-8000-000000000001",
  "event_type": "COMPANY_DISCOVERED",
  "event_version": 1,
  "subject": "company.br.cnpj.discovered.v1",
  "source": "cnpj-data-publisher",
  "occurred_at": "2026-08-05T20:00:00Z",
  "correlation_id": "018f0000-0000-7000-8000-000000000002",
  "causation_id": null,
  "trace_id": null,
  "data": {
    "cnpj": "12345678000190",
    "headquarters_branch": "HEADQUARTERS",
    "legal_name": "EMPRESA EXEMPLO LTDA",
  }
}
```

```

    "trade_name": "EMPRESA EXEMPLO",
    "registration_status": "ACTIVE",
    "registration_status_code": "02",
    "opening_date": "2026-07-10",
    "legal_nature": {
      "code": "2062",
      "description": "SOCIEDADE EMPRESARIA LIMITADA"
    },
    "main_cnae": {
      "code": "6201501",
      "description": "DESENVOLVIMENTO DE PROGRAMAS DE COMPUTADOR"
    },
    "secondary_cnaes": [],
    "company_size_code": "03",
    "share_capital": "100000.00",
    "simple_tax_option": true,
    "mei_option": false,
    "address": {
      "street_type": "RUA",
      "street": "EXEMPLO",
      "number": "100",
      "complement": null,
      "district": "CENTRO",
      "postal_code": "00000000",
      "city_code": "7107",
      "city_name": "SAO PAULO",
      "state": "SP",
      "country_code": "105"
    },
    "contacts": {
      "phones": [
        {
          "area_code": "11",
          "number": "999999999"
        }
      ],
      "email": "contato@empresa.com.br"
    }
  },
  "metadata": {
    "dataset": "RECEITA_FEDERAL_CNPJ",
    "snapshot_version": "2026-07",
    "schema_version": 1,
    "fingerprint": "sha256"
  }
}

```

23. Evento COMPANY_UPDATED

Incluir estado atual e lista de campos alterados.

```
{
  "event_type": "COMPANY_UPDATED",
  "data": {
    "cnpj": "12345678000190",
    "company": {},
    "changes": [
      {
        "field": "contacts.email",
        "previous": "old@empresa.com.br",
        "current": "novo@empresa.com.br"
      },
      {
        "field": "share_capital",
        "previous": "100000.00",
        "current": "150000.00"
      }
    ]
  }
}
```

Não incluir valores anteriores quando campo puder conter dado excessivamente grande.

Limitar `changes`.

Payload completo atual continua disponível em `company`.

24. Evento COMPANY_REACTIVATED

```
{
  "event_type": "COMPANY_REACTIVATED",
  "data": {
    "cnpj": "12345678000190",
    "previous_registration_status": "SUSPENDED",
    "previous_registration_status_code": "03",
    "current_registration_status": "ACTIVE",
    "current_registration_status_code": "02",
    "company": {}
  }
}
```

```
}  
}
```

25. Evento COMPANY_INACTIVATED

```
{  
  "event_type": "COMPANY_INACTIVATED",  
  "data": {  
    "cnpj": "12345678000190",  
    "previous_registration_status": "ACTIVE",  
    "previous_registration_status_code": "02",  
    "current_registration_status": "CLOSED",  
    "current_registration_status_code": "08",  
    "registration_status_date": "2026-07-20",  
    "registration_status_reason": "..."  
  }  
}
```

Não publicar evento de enriquecimento.

26. Evento SNAPSHOT_READY

```
{  
  "event_type": "CNPJ_SNAPSHOT_READY",  
  "subject": "company.br.cnpj.snapshot.ready.v1",  
  "data": {  
    "snapshot_version": "2026-07",  
    "schema_version": 1,  
    "total_rows": 0,  
    "active_rows": 0,  
    "headquarters_rows": 0,  
    "storage": {  
      "type": "S3",  
      "bucket": "cnpj-snapshots",  
      "prefix": "2026-07/active/",  
      "manifest": "2026-07/manifest.json"  
    },  
    "checksum": "sha256"  
  }  
}
```

Quando storage local:

```
não expor caminho interno inutilizável para consumidor externo
```

Nesse caso:

```
desabilitar anúncio de download  
ou  
usar serviço HTTP/S3 para acesso
```

27. Object Storage

Suportar:

```
LOCAL  
S3
```

Configuração:

```
STORAGE_BACKEND=LOCAL  
STORAGE_LOCAL_PATH=/data  
  
S3_ENDPOINT=  
S3_REGION=  
S3_BUCKET=  
S3_ACCESS_KEY=  
S3_SECRET_KEY=  
S3_FORCE_PATH_STYLE=true
```

Compatível com MinIO.

Quando S3 ativo:

```
upload do snapshot  
upload do diff  
upload do manifesto  
upload de rejeitados
```

Usar escrita temporária e promoção final.

28. Scheduler Kubernetes

Criar CronJob:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cnpj-data-publisher-ingestor
spec:
  schedule: "0 4 15 * *"
  timeZone: "America/Sao_Paulo"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: ingestor
              image: cnpj-data-publisher:latest
              args:
                - ingest
                - --snapshot
                - latest
```

Se snapshot do mês ainda não estiver disponível:

```
encerrar com estado SKIPPED
não marcar como falha
tentar novamente no próximo agendamento
```

Permitir agendamento mais frequente:

```
schedule: "0 4 * * *"
```

Ingestor detecta se existe snapshot novo.

29. Docker

Criar Dockerfile multi-stage.

Requisitos:

```
Python 3.12 slim
dependências compiladas em builder
imagem final mínima
tini
usuário não-root
sem segredos
versões fixadas
healthcheck no publisher
labels OCI
```

Exemplo de comandos:

```
docker compose up -d postgres nats
docker compose run --rm app migrate
docker compose run --rm app ingest --snapshot latest
docker compose up -d outbox-publisher
```

Docker Compose deve incluir:

```
PostgreSQL
NATS JetStream
migrations
outbox-publisher
perfil opcional para ingestão
MinIO opcional
```

NATS local:

```
JetStream habilitado
volume persistente
porta cliente
porta monitoramento
```

30. Helm Chart

Chart:

```
helm/cnpj-data-publisher
```

Recursos:

```
CronJob ingestor
Deployment outbox-publisher
Job migrations
Job opcional backfill
ConfigMap
Secret references
ServiceAccount
PVC
PodMonitor opcional
NetworkPolicy opcional
NATS dependency opcional
PostgreSQL dependency opcional
MinIO dependency opcional
```

Configuração base:

```
image:
  repository: ghcr.io/0xai/cnpj-data-publisher
  tag: latest
  pullPolicy: IfNotPresent

ingestor:
  enabled: true
  schedule: "0 4 15 * *"
  timeZone: "America/Sao_Paulo"
  concurrencyPolicy: Forbid

outboxPublisher:
  enabled: true
  replicas: 1

backfill:
  enabled: false

storage:
```

```
backend: LOCAL
local:
  enabled: true
  size: 250Gi
  storageClassName: local-path

postgresql:
  enabled: false

externalPostgresql:
  secretName: cnpj-data-publisher-postgresql
  secretKey: database-url

nats:
  enabled: false

externalNats:
  secretName: cnpj-data-publisher-nats
  urlKey: url
  credentialsKey: credentials

minio:
  enabled: false

resources:
  ingestor:
    requests:
      cpu: "1"
      memory: 2Gi
    limits:
      cpu: "4"
      memory: 8Gi

  outboxPublisher:
    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 512Mi
```

values-k3s.yaml :

```
storage:
  backend: LOCAL
  local:
```

```
    enabled: true
    storageClassName: local-path
    size: 250Gi

nats:
  enabled: true
  jetstream:
    replicas: 1

postgresql:
  enabled: true

outboxPublisher:
  replicas: 1
```

Permitir NATS externo do projeto de enriquecimento.

31. Probes

Outbox publisher deve expor:

```
GET /healthz
GET /readyz
GET /metrics
```

`healthz`:

processo vivo

`readyz`:

```
PostgreSQL acessível
NATS acessível
stream disponível
```

Ingestor é Job. Não precisa probe HTTP.

32. Segurança

Containers:

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL
  seccompProfile:
    type: RuntimeDefault
```

Exceções:

```
PVC de dados gravável
/tmp via emptyDir
```

NATS:

```
credencial exclusiva
publish-only
sem permissão de subscribe
subjects restritos a company.br.cnpj.*
```

PostgreSQL:

```
usuário exclusivo
senha via Secret
SSL configurável
```

Download:

```
allowlist de host
HTTPS preferido
timeout
limite de tamanho
sem execução de shell
```

Não registrar:

```
senhas
tokens
NATS credentials
S3 secret
DATABASE_URL completo
```

33. Observabilidade

Logs JSON.

Exemplo:

```
{
  "timestamp": "2026-08-05T20:00:00Z",
  "level": "INFO",
  "service": "cnpj-data-publisher",
  "component": "ingestor",
  "event": "snapshot_diff_completed",
  "snapshot_version": "2026-07",
  "previous_snapshot_version": "2026-06",
  "discovered": 1000,
  "updated": 500,
  "reactivated": 20,
  "inactivated": 200,
  "duration_ms": 120000
}
```

Métricas:

```
cnpj_snapshot_runs_total
cnpj_snapshot_run_duration_seconds
cnpj_download_bytes_total
cnpj_download_failures_total
cnpj_rows_processed_total
cnpj_rows_rejected_total
cnpj_active_companies_total
cnpj_diff_companies_total
cnpj_outbox_pending_total
cnpj_outbox_published_total
cnpj_outbox_failed_total
```

```
cnpj_backfill_rows_total  
cnpj_backfill_progress  
cnpj_nats_publish_duration_seconds
```

Labels de baixa cardinalidade:

```
status  
event_type  
snapshot_version  
component
```

Não usar CNPJ como label Prometheus.

34. Retenção

Configuração:

```
KEEP_RAW_DOWNLOADS_MONTHS=2  
KEEP_EXTRACTED_FILES_MONTHS=1  
KEEP_CANONICAL_SNAPSHOTS_MONTHS=3  
KEEP_DIFFS_MONTHS=6  
KEEP_REJECTED_ROWS_MONTHS=3  
KEEP_PUBLISHED_OUTBOX_DAYS=14
```

Nunca apagar:

```
snapshot atual  
snapshot anterior necessário para diff  
snapshot usado por backfill ativo  
arquivos com execução FAILED até diagnóstico
```

Criar comando:

```
cnpj-data-publisher cleanup
```

35. Concorrência e locks

Somente uma ingestão por vez.

Usar:

```
PostgreSQL advisory lock
```

Lock:

```
cnpj-data-publisher-ingest
```

Outbox publisher permite múltiplas réplicas usando:

```
FOR UPDATE SKIP LOCKED
```

Backfill possui lock por `backfill_run_id`.

36. Tratamento de falhas

Download falhou

```
retry  
preservar partial  
retomar quando possível  
marcar execução FAILED após limite  
não alterar snapshot atual
```

Parsing falhou

```
registrar linha rejeitada  
continuar até limite  
falhar quando percentual exceder configuração
```

DuckDB falhou

```
preservar banco temporário  
não promover snapshot  
registrar query e estágio
```

PostgreSQL indisponível

falhar ingestão antes de gerar eventos
outbox publisher aguarda e reconecta

NATS indisponível

eventos ficam no outbox
publisher aplica retry
ingestão pode terminar com sucesso

Publisher caiu após PubAck

Possível publicação duplicada.

Proteções:

Nats-Msg-Id
duplicate_window
event_id estável
consumidor idempotente

Não prometer exatamente uma vez.

Garantia:

at-least-once

37. Testes unitários

Cobrir:

descoberta de snapshots
manifest parsing
download resume
SHA-256
Zip Slip
layout dos arquivos

```
datas
capital social
CNPJ textual
normalização
situação cadastral
fingerprint determinístico
diff
event schemas
outbox retry
backfill cursor
retenção
```

38. Testes de integração

Usar containers reais:

```
PostgreSQL
NATS JetStream
MinIO opcional
```

Testar:

```
migrations
outbox publication
PubAck
retry
reconexão NATS
FOR UPDATE SKIP LOCKED
múltiplos publishers
S3 upload
backfill resumível
```

39. Teste E2E

Criar fixtures pequenas:

```
snapshot-2026-06
snapshot-2026-07
```

Cenários:

```
empresa nova ativa  
empresa inalterada  
empresa atualizada  
empresa reativada  
empresa inativada  
empresa nova inativa  
linha inválida  
capital social inválido  
data inválida  
evento duplicado  
NATS temporariamente indisponível  
restart durante backfill
```

Teste completo:

```
fixtures ZIP  
↓  
download simulado  
↓  
extração  
↓  
normalização  
↓  
Parquet  
↓  
diff  
↓  
outbox  
↓  
NATS JetStream  
↓  
consumer de teste  
↓  
validação JSON Schema
```

Critério:

```
empresa inalterada não gera evento
```

40. Teste de performance

Gerar dataset sintético.

Cenários:

```
1 milhão de registros  
5 milhões de registros  
10 milhões de registros
```

Medir:

```
tempo de parsing  
uso máximo de memória  
tempo de criação Parquet  
tempo de diff  
eventos por segundo  
tamanho de storage
```

Processamento deve ser streaming ou por lotes.

Não permitir uso de memória proporcional ao dataset completo.

41. CI

GitHub Actions:

```
ruff  
format check  
mypy ou pyright  
unit tests  
integration tests  
E2E  
Docker build  
Trivy  
Helm lint  
Helm template  
JSON Schema validation  
contract compatibility check
```

Pipeline:

```
pull_request
push main
tag
```

Publicação de imagem somente em tag ou configuração explícita.

Tags:

```
sha-<commit>
v1.0.0
latest
```

42. Makefile

```
make setup
make lint
make format
make typecheck
make test
make test-unit
make test-integration
make test-e2e
make test-performance
make build
make up
make down
make migrate
make ingest
make backfill
make cleanup
make helm-lint
make helm-template
make k3s-install
make k3s-upgrade
make k3s-uninstall
```

Exemplos:

```
make ingest SNAPSHOT=2026-07
make backfill SNAPSHOT=2026-07 RATE=100
make k3s-install NAMESPACE=cnpj-data
```

43. README

Documentar:

```
objetivo
arquitetura
fonte dos dados
estrutura dos snapshots
contratos NATS
execução local
execução Docker
deploy K3s
configuração NATS externo
configuração PostgreSQL externo
configuração MinIO
ingestão manual
backfill
retry
retenção
observabilidade
troubleshooting
```

Adicionar diagrama Mermaid:

```
flowchart LR
    RF[Receita Federal] --> ING[Ingestor]
    ING --> DDB[DuckDB]
    DDB --> PQ[Parquet Snapshots]
    PQ --> DIFF[Diff Engine]
    DIFF --> OUTBOX[(PostgreSQL Outbox)]
    OUTBOX --> PUB[Outbox Publisher]
    PUB --> NATS[NATS JetStream]
    NATS --> CONSUMERS[Consumers]
```

44. Variáveis de ambiente

Criar `.env.example`.

```
APP_ENV=development
LOG_LEVEL=INFO

DATABASE_URL=postgresql+psycpg://user:password@postgres:5432/cnpj

NATS_URL=nats://nats:4222
NATS_CREDS_FILE=
NATS_STREAM=BRAZIL_COMPANY_EVENTS
NATS_SUBJECT_PREFIX=company.br.cnpj

RECEITA_BASE_URL=
RECEITA_INCLUDE_PARTNERS=false
RECEITA_MAX_DOWNLOAD_CONCURRENCY=4
RECEITA_REQUEST_TIMEOUT_SECONDS=60

STORAGE_BACKEND=LOCAL
STORAGE_LOCAL_PATH=/data

S3_ENDPOINT=
S3_REGION=
S3_BUCKET=
S3_ACCESS_KEY=
S3_SECRET_KEY=
S3_FORCE_PATH_STYLE=true

INCLUDE_BRANCHES=false
INITIAL_SNAPSHOT_MODE=STORE_ONLY

OUTBOX_BATCH_SIZE=500
OUTBOX_CONCURRENCY=20
OUTBOX_MAX_ATTEMPTS=20

PARQUET_COMPRESSION=zstd
PARQUET_ROW_GROUP_SIZE=250000

MAX_REJECTED_ROW_PERCENTAGE=0.01

KEEP_RAW_DOWNLOADS_MONTHS=2
KEEP_EXTRACTED_FILES_MONTHS=1
KEEP_CANONICAL_SNAPSHOTS_MONTHS=3
KEEP_DIFFS_MONTHS=6
KEEP_PUBLISHED_OUTBOX_DAYS=14
```

```
METRICS_PORT=9090
HEALTH_PORT=8080
OTEL_ENABLED=false
OTEL_EXPORTER_OTLP_ENDPOINT=
```

45. Critérios de aceite

Implementação considerada concluída quando:

1. `docker compose up` sobe PostgreSQL, NATS e publisher.
2. Migration executa sem intervenção manual.
3. Fixture E2E gera snapshot Parquet válido.
4. Segundo snapshot gera diff correto.
5. Empresa inalterada não gera evento.
6. Empresa nova ativa gera `COMPANY_DISCOVERED`.
7. Empresa alterada ativa gera `COMPANY_UPDATED`.
8. Empresa reativada gera `COMPANY_REACTIVATED`.
9. Empresa inativada gera `COMPANY_INACTIVATED`.
10. Eventos passam por PostgreSQL Outbox.
11. Eventos chegam no NATS JetStream.
12. Eventos validam contra JSON Schema.
13. Falha no NATS não perde eventos.
14. Restart do publisher não perde eventos.
15. Backfill pode parar e continuar.
16. Primeiro snapshot usa `STORE_ONLY` por padrão.
17. Helm Chart passa em `helm lint`.
18. Helm template funciona com `values-k3s.yaml`.
19. Containers rodam sem root.
20. Métricas e logs estruturados disponíveis.
21. README permite instalação completa sem conhecimento externo.
22. Nenhum código de enriquecimento existe no projeto.

46. Ordem de implementação

Fase 1

```
estrutura do projeto
configuração
logging
Docker
```

PostgreSQL
migrations

Fase 2

discovery
download
manifesto
extração
fixtures

Fase 3

layouts
parsing
normalização
DuckDB
Parquet

Fase 4

fingerprints
diff
manifestos de diff

Fase 5

contratos JSON Schema
event outbox
outbox publisher
NATS JetStream

Fase 6

backfill
rate limiting
checkpoint
resume

Fase 7

```
métricas  
health endpoints  
retenção  
segurança
```

Fase 8

```
Helm Chart  
K3s  
CI  
testes E2E  
documentação
```

47. Restrições finais

Não entregar pseudocódigo como resultado final.

Entregar:

```
código executável  
migrations  
Dockerfile  
docker-compose.yml  
Helm Chart  
JSON Schemas  
fixtures  
testes  
Makefile  
README  
CI
```

Não criar dependência direta com ShadowTrace.

Não importar código do projeto de enriquecimento.

Não compartilhar banco com projeto consumidor.

Não usar fila em memória.

Não depender de disco efêmero.

Não publicar milhões de eventos automaticamente no primeiro deploy.

Não reenviar empresas inalteradas todo mês.

Não assumir CNPJ exclusivamente numérico.

Não substituir snapshot válido por snapshot incompleto.

Não perder eventos quando NATS estiver indisponível.

Resultado esperado:

```
serviço genérico  
independente  
reutilizável  
event-driven  
resumível  
observável  
seguro  
pronto para Docker  
pronto para K3s
```